MyCodeSchool

Code review — recapitulare-initiere-cpp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 4 — 2026-10-15 — sha 6c66dca

Am compilat fiecare header separat și am rulat fiecare funcție pe exemplele din fișă, plus capetele. Runda 3 nu ți-a ajuns; ce a rămas deschis din ea e mai jos.

Regula, de acum: `algoritmi.h` e **doar de studiu**. Îl citești, îl închizi, scrii tu algoritmul în headerul tău. **Nu-l incluzi, nu chemi nimic din el, nu copiezi din el.** Caseta din fișă spunea să-l incluzi — acolo am greșit eu.

Poarta de compilare: 🔴 pică — vezi B17.

Fișa, secțiunile 1–5:

✅ corecte și scrise de tine	2.2 · 3.5
🟡 corecte, dar pe funcții din `algoritmi.h`	1.1 · 1.2 · 1.3 · 3.1 · 3.2 · 4.1 · 5.1 · 5.4
🔴 rezultat greșit	1.4 · 2.1 · 2.3 · 2.4 · 3.4 · 4.2 · 4.4 · 5.2 · 5.3 · 5.5
⏳ neterminate	1.5 · 2.5 · 3.3 · 4.3 · 4.5

Închise din rundele trecute: B2 (în **toate** cele 3 locuri — ai reparat regula, nu doar linia), B5, B7, B9, B11, B14.

🔴 Critice

B17 — 4 headere depind de `algoritmi.h`, și acum nu mai compilează `cautare_binara.h:4` · `minim_maxim.h:4` · `sortare_bule.h:4` · `sortare_selectie.h:4` · `algoritmi.h:169`

```
algoritmi.h:169: error: use of undeclared identifier 'aux'
```

Ai mutat o linie în `algoritmi.h`. Cele 4 headere îl includ, deci pică toate. `#include` lipește textul fișierului în al tău: o linie stricată acolo strică tot ce o include. Plus, `cautare_binara.h:203–206` are 3 erori proprii (`sol25`), iar `acf2773` și `8762a7b` nu compilau nici ele.

💡 Scoate `#include "algoritmi.h"` din toate 4. Ce funcții îți lipsesc acum? Pe acelea le scrii tu, sus în fiecare header. 💡 Înainte de commit: fiecare header, pe rând, în `main.cpp`, apoi F9.

B18 — interschimbarea în ordinea greșită (5.2, 5.3) `sortare_interval.h:61` · `:70–71`

```
5.2  ->  9 7 5 6 6 6            trebuia 9 7 5 3 2 1
5.3  ->  5 7 9 16777216 ...     trebuia 5 7 9 3 2 1
```

`v[i + 1] = aux` e scris **înainte** ca `aux` să primească ceva. Iar `int aux;` de la `:61` doar a făcut eroarea să tacă: acum citești un `aux` fără valoare, de unde vin `6` și `16777216`.

💡 Trasează pe hârtie o interschimbare între `5` și `3`. Ce valoare trebuie salvată **prima**, ca să n-o pierzi? 💡 Când compilatorul zice `undeclared`, nu declara variabila ca să tacă — întreabă-te de ce n-o vede.

B19 — funcție `int` fără `return`, de 7 ori `cautare_binara.h:90`, `:108`, `:190` · `minim_maxim.h:62` · `sortare_bule.h:43` · `sortare_selectie.h:24`, `:44`

```
pozitiaDeInserat(2 5 9 14, pun 20)
runtime error: execution reached the end of a value-returning function
```

Cine cheamă funcția citește rezultatul oricum; dacă n-ai scris `return`, citește gunoi (`sol4()` afișează `0`).

💡 Funcția doar afișează? Atunci ce tip ar trebui să aibă? 💡 Funcția trebuie să dea un răspuns? Ce răspuns dai când bucla se termină fără să găsească nimic? Exemplul `pun 20 -> 4` din fișă îți spune.

B20 — 2.4: `<` și `>` în loc de `<=` și `>=` `cautare_binara.h:142`, `:145`

```
x = 9  -> -1    trebuia 9
x = 1  ->  5    trebuia 2
```

Exemplele din fișă trec; capetele pică. `minim < maxim` nu mai verifică ultimul element rămas; `>` sare peste valoarea egală.

💡 Uită-te la `cautarePrimaPoz`, la 30 de linii mai jos. Tu ai scris-o, și e corectă. Ce face ea, față de ce cere 2.4?

B21 — 2.1: căutare binară pe un vector neordonat `cautare_binara.h:48`, `:57`

```
30 12 45 7 19, caut 19  ->  0    trebuia 1
```

Căutarea binară aruncă o jumătate fără s-o verifice — are voie doar dacă vectorul e ordonat. Fișa scrie „nu este neapărat ordonat”: asta e capcana. În `sol21` ai și `dim = 4` pentru 5 valori.

💡 Trasează căutarea lui `19`: ce e la mijloc, ce jumătate arunci? 💡 Ce trebuie să faci cu vectorul **înainte** de căutare?

B22 — 1.4: compari o valoare cu o poziție `minim_maxim.h:64–69`

`pozMaxim` dă **unde** e maximul, nu **cât** e. Pe `3 3 8 1` răspunsul iese `3` din întâmplare — e poziția lui `1`, nu a doua valoare.

💡 La 1.1 ai scris `v[pozMaxim(v, dim)]`. De ce acolo ai pus `v[...]` în jur?

B23 — 3.4: funcția care întreabă „e ordonat?” îl ordonează `sortare_bule.h:67`

```
dejaOrdonat(1 7 3) -> 0   corect, dar vectorul după apel: 1 3 7
```

Ai sortat originalul, nu copia. Un vector trimis unei funcții nu e copiat: ce schimbi înăuntru rămâne schimbat și afară.

💡 Fără sortare: ce trebuie să fie adevărat pentru **fiecare pereche de vecini** ca vectorul să fie crescător?

B24 — 4.2: afișezi ultimele cifre, nu numerele `sortare_selectie.h:26–35`

```
23 41 15 32  ->  1 2 3 5      trebuia 41 32 23 15
```

După ce ai sortat cifrele separat, nu mai știi cui aparține fiecare.

💡 În sortarea prin selecție, **ce compari** și **ce muți** pot fi lucruri diferite.

B25 — 4.4: `v[i - 1]` la `i = 0` `sortare_selectie.h:49`

Afișează corect, din noroc: la `i = 0` citește `v[-1]`, memorie din afara vectorului (AddressSanitizer: `stack-buffer-underflow`).

💡 Primul element are vecin în stânga?

B26 — 5.5: prima treime pornește de la 1 `sortare-interval.h:133`

```
9 7 5 3 1 2  ->  9 7 3 5 1 2     trebuia 7 9 3 5 1 2
```

💡 De la ce poziție începe un vector?

B27 — funcții copiate identic din `algoritmi.h` `cautare_binara.h:108` `inserareInVector` ≡ `inserareElement` · `sortare-interval.h:40` `sortareInterval` ≡ `sortareInterval` · `sortare-interval.h:59` `sortareIntervalDescrescator` (copiată, apoi stricată — B18)

O funcție copiată nu e rezolvată. Comparat caracter cu caracter, fără spații.

💡 Citește algoritmul, închide fișierul, scrie-l din memorie. Dacă te blochezi, trasează-l pe hârtie — tu faci asta deja foarte bine.

B15 — `produsCifre(234)` **→ 4, trebuia 24** (din runda 3) `parcurgerea-vectoriilor.h:183`

`suma * n % 10` : `*` și `%` au aceeași prioritate și se fac de la stânga la dreapta.

💡 Unde pui paranteza ca `% 10` să se aplice doar lui `n` ?

B16 — „prime între ele" ≠ „amândouă prime" (din runda 3) `parcurgerea-vectoriilor.h:223`

`8` și `9` sunt prime între ele, deși niciunul nu e prim.

💡 Ce funcție ai deja (și ai reparat-o chiar acum) care spune dacă două numere au divizori comuni?

🟡 Importante

- **M1** — garda cu cratimă: `sortare-interval.h:1` (și tot în `while-ex.h:1` , `parcurgerea-vectoriilor.h:1`). În celelalte 4 ai scris-o bine.
- **M14** — `mediana` , `kValoare` , `primeleK` sortează vectorul primit. Același mecanism ca B23.
- **M16** — fișiere goale comise: `interclasare.h` , `munte_vale.h` , `stergere.h` , `sortate_bule.h` , `fisa2.h` .
- **M4** — `cmmmc(50000, 70000)` → `-79496` . 💡 Ce se întâmplă dacă împarți înainte să înmulțești?
- **M17** — `cifraMaxima(0)` → `-1` . 💡 Câte cifre are `0` ?
- **Restanțe r1** (`while-ex.h` , funcții pe care nu le mai chemi): B3, B4, B6, M2, M3, M6 — după ce termini secțiunile 1–5.

🟢 Cleanups

- **C20** — `afisareDistinceDescrescatoare` afișează crescător; `celMaiMicNrCuSumaCifrelorGresit` e varianta **corectă**.
- **C21** — vectori `v` declarați și nefolosiți în `sol1`, `sol2`, `sol31`, `sol34`.
- **C22** — `primeleK` cu `k > dim` citește dincolo de vector.
- **C23** — mesaje de commit: `clasa` → „5.1–5.5 sortare pe interval".
- **C18** — `bin/`, `obj/` tot în git.

Ce e bine

- **B2 reparat peste tot**, nu doar unde am arătat eu. Fă la fel cu fiecare.
- `cautarePrimaPoz` **și** `cautareBinara1` — căutări binare scrise de tine, corecte pe toate capetele.
- `sortarePeInterval` inversează `a` și `b` dacă vin invers. Nu ți-a cerut-o nimeni.
- `cateInterschimbari` (3.5) corectă: `3 1 2` → 2, `1 2 3` → 0.
- **Trasările pe hârtie** (`sortare-interval.h:8-38`). Trasează și funcția **ta**, nu doar pe cea din material — vedeai B18 la primul pas.

Q&A — răspunde în scris, înainte de runda 5

1. De unde vin numerele `16777216` din 5.3?

2. Pe `30 12 45 7 19`, căutarea binară îl găsește pe `30`, dar nu pe `19`. De ce?

3. Pe `3 3 8 1`: ce dă `pozMaxim(v, 4)` și ce dă `v[pozMaxim(v, 4)]`?

Runda 3 — 2026-10-01 — sha ee7010e (condensată, nelivrată la timp)

Generată pe 2026-10-01, n-a ajuns la tine. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B15	`parcurgerea:183`	`produsCifre`: `suma * n % 10` → mereu o cifră	🔴 NEATINS (vezi runda 4)

#	Loc	Constatare	Stare pe `6c66dca`
B16	`parcurgerea:223`	„prime între ele” verificat cu `isPrim`	🔴 NEATINS (vezi runda 4)
B9	`parcurgerea:55–57`	`isPrim(i)` în loc de `isPrim(v[i])`	✅ ÎNCHIS
M10	`parcurgerea:109`	`afisareRasturnat` testează doar capetele	✅ ai ales palindromul: `punctul_c` cu `ePalindrom`
M11	`parcurgerea:156`	`sol1d` nu poate afișa ramura adevărată	🟡 NEATINS
M12	`parcurgerea:20`, `:253`	Cod comentat care nu compilează	🟡 `punctul_c` rescris ✅; `sol1` și `punctul_g` tot comentate
M7	—	`isPrim` ≡ `estePrim`, `CifraMinima` ≡ `cifraMinima`	🟡 NEATINS
C14	`parcurgerea:107`, `:207`	Variabile nefolosite	🟢 NEATINS
C15	`parcurgerea:168`	`?:` cu `<<`	🟢 NEATINS
C16	`parcurgerea:142`	`divizoriProprii` întoarce un număr	🟢 NEATINS
C17	—	`soll2`	✅ redenumit `sol2`
C18	`bin/`, `obj/`	Binare în git	🟢 NEATINS

Runda 2 — 2026-10-01 — sha fdda59a (condensată)

Textul integral e în PDF-ul rundei 2 (commit `b3b1bf6`).

#	Constatare	Stare pe `6c66dca`
B8	Nu compila, 7 erori	✅ ÎNCHIS (runda 3)
B9	`contorPrimeVector` număra indicii	✅ ÎNCHIS
B10	Indexare de la 1 vs de la 0	✅ ÎNCHIS
B11	Palindromul testa doar capetele	✅ ÎNCHIS
B12	`punctul_g` — buclă infinită	✅ ÎNCHIS
B13	`if` în afara buclei	✅ ÎNCHIS
B14	Funcție `int` fără `return`	✅ `afisareaElemVect` e `void` · 🔴 tiparul revine în runda 4 (B19)
M7	Funcții rescrise, existente deja	🟡 NEATINS · copiere în runda 4 (B27)
M8	`int c[10]` cu indexare de la 1	✅ codul a fost rescris
M9	`punctul_g` folosește `nr` nesetat	🟡 cod comentat
C7	`CIfraMinima`	✅ `CifraMinima`
C8	`//todo:` învechit	✅
C9	`dim` declarat, literal pasat	🟢 NEATINS (`parcurgerea:69`)
C10, C11	`main.cpp` vechi	✅ `main.cpp` rescris

#	Constatare	Stare pe `6c66dca`
C12	Binare în git	🟢 NEATINS
C13	Mesaje de commit	🟢 NEATINS

Runda 1 — 2026-09-24 — sha 2734bb4 (condensată)

Livrată pe 2026-10-01, odată cu runda 2. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B1	`main.cpp:1`	`while-ex.h` nu ajungea la compilator	✅ compilat separat de mine: 0 erori
B2	`while-ex.h:72`, `:429`, `:352`	Cifra scoasă înainte de `while`	✅ ÎNCHIS în toate cele 3 locuri
B3	`while-ex.h:126`, `:171`	Două bucle cu `n` nemodificat → nu se termină	🔴 NEATINS
B4	`while-ex.h:124`, `:169`	Funcții `int` fără `return`	🔴 2 rămase · vezi B19
B5	`while-ex.h:303`	`cmmdc(0, 0)` → împărțire la zero	✅ ÎNCHIS
B6	`while-ex.h:452`	`eliminareCifra` scoate toate aparițiile	🔴 NEATINS
B7	`while-ex.h:370`	`celMaiMicNrCuSumaCifrelor(20)` → 99	✅ ÎNCHIS (funcția nouă)
M1	gărzile `.h:1`	Cratimă în numele macro-ului	🔴 NEATINS + `sortare-interval.h`

#	Loc	Constatare	Stare pe `6c66dca`
M2	`while-ex.h:154`, `:201`	`% p` în loc de `% 10`	🔴 NEATINS
M3	`while-ex.h:479`, `:554`	`pow` fără `<cmath>`	🔴 NEATINS
M4	`while-ex.h:345`	`a * b` → depășire	🔴 reprodus: `cmmmc(50000,70000)` → `-79496`
M5	`fisa2.h`	Fișier gol	🔴 NEATINS · vezi M16
M6	toate `.h`	Definiții în `.h`	🟡 NEATINS
C1–C3	—	`dim`/literal, `int n` nefolosit, `isZero`	🟢 NEATINSE
C4	`bin/`, `obj/`	Binare în git	🟢 NEATINS
C5	`while-ex.h:8`, `main.cpp:3`	`//todo:` învechite	🟢 `main.cpp` ✅, `while-ex.h:8` rămas
C6	—	Mesaje de commit	🟢 NEATINS